EDGE

TELIT CINTERION | UPPAY

Software Documentation

List Of Contents

1. Overview 3
2. Building Software 3
   2.1. Host PC Requirements 3
   2.2. Downloading Software and Patches 3
   2.3. Creating Virtual Environment 3
   2.4. Preparing Source Code 4
   2.5. Building the Software 6
      2.5.1. Building individual software components 6
3. Flashing the device 6
   3.1. Flashing through QFIL 6
   3.2. Flashing through fastboot 7
4. Peripherals 7
   4.1. Serial Interfaces 7
   4.2. Display Interface 8
   4.3. Touchscreen Interface 9
   4.4. NFC Interface 9
   4.5. Coin Pulse Interface 10
   4.6. Module power state management 10
      4.6.1. Flashing the attiny 10

List Of Tables

List of Figures

Figure 1 MDB to Serial conversion 8

1. Overview

This document contains the Uppay UpDroid payment Kiosk software compilation guide, flashing guide and peripheral overview. This document is complementary to the vendor documentation, specifically the *SE250B4_SDK_User_Guide_r1*.

2. Building Software

The software package running on the Application Processor (AP) is referred to as the HLOS package, which includes the operating system, user interface application, and other necessary components. The SDK package provides the source code for the following Non-HLOS software subsystems, including:

- ADSP subsystem (SE250B4 module) – Available in the ADSP.VT.5.4.1 package.
- Bootloader source code – Provided in the BOOT.XF.4.1 package.
- TrustZone subsystem – Available in the TZ.XF.5.1 package.

2.1. Host PC Requirements

Component	Minimum	Recommended
Memory	32 GB	64 GB
Storage	512 GB	1 TB
Host OS	Ubuntu 14.04 LTS (64 bit)	

2.2. Downloading Software and Patches

The following packages are proprietary and only available though the manufacturer.

- The `SE250B4_ENV.tar.gz` package provides the portable build environment required to build SE250B4 SDK source code.
- The `SE250B4_42.xx.xxx.tar.gz` Release package contains patch files for both HLOS and Non-HLOS subsystem's source codes.
- The `uppay-cell-module-sw.zip` Release package contains the sdk changes specific to the UPDroid device.

2.3. Creating Virtual Environment

For host OS independent development it is recommended the creation of a LXC container image that will contain all the required software to build this system. In this demonstration we are using Ubuntu as the Host OS, in cases of non Debian based distros the user will need to search for the equivalent command for installing the needed packages.

To setup the portable environment, follow these steps:

1) Install lxc

```
sudo apt install -y lxd
```

2) Initialize the program

```
lxd init
```

3) Create a Ubuntu 14.04 LTS image

```
lxc launch images:ubuntu/14.04 my-ubuntu14-container
```

4) Enter the container

```
lxc exec my-ubuntu-14-container -- bash
```

5) Update the container

```
sudo apt-get update
```

6) Install the required libs

```
sudo apt-get install git-core gnupg flex bison build-essential zip curl
zlib1g-dev libc6-dev-i386 x11proto-core-dev libx11-dev lib32z1-dev libgl1-
mesa-dev libxml2-utils xsltproc unzip fontconfig repo schroot
```

7) Configure a GitHub account

```
sudo git config --global user.email "your@email.com"
sudo git config --global user.name "your name"
sudo git config --global http.followRedirects true
```

"This email will be used to create a account with Qualcomm and it will be required to sign a NDA for access to the QCS2290 chipcode"

2.4. Preparing Source Code

The `SE250B4` source code consists of the following components:

- Qualcomm QCS2290 Source Code
- AOSP Source Code
- Telit Custom Patches
- EDGE Custom Patches

Steps to Prepare the Source code

1) Create a build directory on the home folder

```
mkdir -p ~/se250b4_build
cd ~/se250b4_build
```

2) Place all software components inside this folder

```
mv ~/SE250B4_ENV.tar.gz ~/se250b4_build/
mv ~/SE250B4_42.xx.xxx.tar.gz ~/se250b4_build/
mv ~/uppay-cell-module-sw.zip ~/se250b4_build/
```

3) Extract the environment package into /srv/chroot

```
sudo mkdir -p /srv/chroot
sudo tar -zxvf ~/se250b4_build/SE250B4_ENV.tar.gz --directory=/srv/chroot/
```

4) Modify the schroot configuration file in "/srv/schroot.conf"

```
...
[se250b4]
Description=se250b4 build environment
Type=directory
directory=/srv/chroot/se250b4
users=<user_name>
groups=<user_name>
...
```

5) Modify the schroot fstab file in "/etc/schroot/default/fstab"

```
# fstab: static file system information for chroots.
# Note that the mount point will be prefixed by the chroot path
# (CHROOT_PATH)
```

```
# <file system> <mount point> <type> <options>   <dump>  <pass>
/proc           /proc          none   rw,bind     0       0
/sys            /sys           none   rw,bind     0       0
/dev            /dev           none   rw,bind     0       0
/dev/pts        /dev/pts       none   rw,bind     0       0
/tmp            /tmp           none   rw,bind     0       0

# It may be desirable to have access to /run, especially if you wish
# to run additional services in the chroot. However, note that this
# may potentially cause undesirable behaviour on upgrades, such as
# killing services on the host.
/run            /run           none   rw,bind     0       0
/run/lock       /run/lock      none   rw,bind     0       0
/dev/shm        /dev/shm       none   rw,bind     0       0
/run/shm        /run/shm       none   rw,bind     0       0
```

6) Modify the "/etc/schroot/default/copyfiles" to copy the *resolve* and *local-time* settings into chroot environment

```
...
/etc/resolve.conf
/etc/localtime
...
```

7) Download the QCS2290 source code

```
git clone -b r3.0_00018.0 --depth 1 https://qpm-git.qualcomm.com/home2/git/
qualcomm/snapdragon-mid-2020-iot-spf-3-0_ap_standard_oem.git
```

8) Extract and move the source code into the build folder

```
unzip snapdragon-mid-2020-ipt-spf-3-0_ap_standard_oem.zip
mv snapdragon-mid-2020-ipt-spf-3-0_ap_standard_oem se250b4
```

9) Download LA.QSSI.11.0 source code

```
cd ~/se250b4_build/se250b4/LA.QSSI.11.0/LINUX/android
repo init --depth=1 --current-branch -u https://git.codelinaro.org/clo/la/
la/system/manifest.git -b release -m
AU_LINUX_ANDROID_LA.QSSI.11.0.R1.11.00.00.668.231.00.xml --repo-url=https://
git.codelinaro.org/clo/tools/repo.git --repo-branch=qc-stable
repo sync
```

10) Download LA.QSSI.13.0 source code

```
cd ~/se250b4_build/se250b4/LA.QSSI.13.0/LINUX/android
repo init --depth=1 --current-branch -u https://git.codelinaro.org/clo/la/
la/system/manifest.git -b release -m
AU_LINUX_ANDROID_LA.QSSI.13.0.R1.13.00.00.913.144.00.xml --repo-url=https://
git.codelinaro.org/clo/tools/repo.git --repo-branch=qc-stable
repo sync
```

11) Download LA.UM.9.15.2 source code

```
cd ~/se250b4_build/se250b4/LA.UM.9.15.2/LINUX/android
repo init --depth=1 --current-branch -u https://git.codelinaro.org/clo/la/
la/vendor/manifest.git -b release -m
AU_LINUX_ANDROID_LA.UM.9.15.2.R1.11.00.00.879.106.xml --repo-url=https://
git.codelinaro.org/clo/tools/repo.git --repo-branch=qc-stable
repo sync
```

"The repo sync command accepts the -j argument followed by a integer with the number of threads your processor possesses, this lets the sync happen much faster."

12) Extract and move the telit patches into the build folder

```
tar -zxvf SE250B4_42.00.001_B007.tar.gz
cp -rf se250B4_42.00.001_B007/se250b4/telit ~/se250b4_build/se250b4/
```

13) Apply the Telit patch

```
cd ~/se250b4_build/se250b4/telit
./se250b4.sh
```

14) Apply EDGE patch

```
cp -rf ~/se250b4_build/uppay-cell-module-sw/* ~/se250b4_build/se250b4/
```

2.5. Building the Software

The SE250B4 source code provides the scripts to build HLOS and other non-HLOS subsystems such as BOOT, ADSP and TZ.

Steps to build the software

1) Enter the portable environment

```
schroot -c se250b4
```

2) Execute the build command in the scripts folder

```
cd ~/se250b4_build/se250b4/build_scripts
./build_system_ext.sh --model=0UE0 --build=ALL
```

2.5.1. Building individual software components

Individual software packages can be built by selecting appropriate model and build options.

In the scripts folder:

```
cd ~/se250b4_build/se250b4/build_scripts
```

Build only the Bootloader image

```
./build_system_ext.sh --model=0UE0 --build=BOOT
```

Build only the ADSP image

```
./build_system_ext.sh --model=0UE0 --build=ADSP
```

Build only the TrustZone image

```
./build_system_ext.sh --model=0UE0 --build=TZ
```

Build only the HLOS image

```
./build_system_ext.sh --model=0UE0 --build=HLOS
```

3. Flashing the device

3.1. Flashing through QFIL

1) Build the metadata images required by the QFII tool:

```
cd ~/se250b5_build/se250b4/build_scripts
./meta_build.sh
```

2) Open a shell in the built images folder

```
cd ~/se250b5_build/se250b4/built_images
```

3) Enter QFIL Download mode, by turning the device on while the USB BOOT button is pressed
4) Flash the device using the qdl executable:

```
qdl --storage emmc prog_dddr raw_unparse.xml patch0.xml
```

3.2. Flashing through fastboot

To flash the device through fastboot, connect the device to a PC via an USB cable, then:

1) Open a shell in the built images folder

```
cd ~/se250b5_build/se250b4/built_images
```

2) Enter fastboot mode, executing the following command

```
adb reboot bootloader
```

3) Flash each system image partition individually

```
fastboot flash boot "boot.img"
fastboot flash recovery "recovery.img"
fastboot flash vbmeta_system "vbmeta_system.img"
fastboot flash vbmeta "vbmeta.img"
fastboot flash dtbo "dtbo.img"
fastboot flash super "super.img"
fastboot continue
```

4) Wipe userdata partition

```
fastboot -w
```

5) Boot into the system

```
fastboot continue
```

4. Peripherals

This device contains a series of interfaces for ease of access to its peripherals, most of them already work with the system automatically.

4.1. Serial Interfaces

The Telit SE250B4 module provides three uart interfaces, that can be accessed by its tty device directly or by a framework service called **TelitManagerService**, accessible through tty devices, the following table shows which tty instance is connected to each serial port. In both cases only privileged access is allowed.

Serial Port	RX Label	TX Label	TTY Label	QUP instance
MDB	MDB_RX	MDB_TX	ttyHS0	qupv3_se1_4uart
RS-232	RS232_RX	RS232_TX	ttyHS1	qupv3_se4_4uart
TTL	TTL_RX	TTL_TX	ttyHS2	qupv3_se5_4uart

These serial interfaces can be accessed directly by an application running as root in the system, or indirectly by using a custom Android framework service added by the SoM vendor, only applications trusted by the system will have access to this API. A sample application is available in the source repository showcasing the API in action, the demo application is called **TelitManager**. The RS-232 and TTL serial devices by default work with 9600 baudrate, 8 bits of data, no parity, 1 stop bit and no flow control. Whereas the MDB serial have an extra 9th bit used to distinguish between addresses and data messages, in our implementation the attiny826 microcontroller act as a converter between a 8

bit serial port of the SE250B4 Module and an external MDB serial, the communication between the 8 bits serials make use of a special character, the '$' or 24 in hexadecimal, that is used to signal that the next byte is a address byte, the image bellow demonstrates how this scheme works.

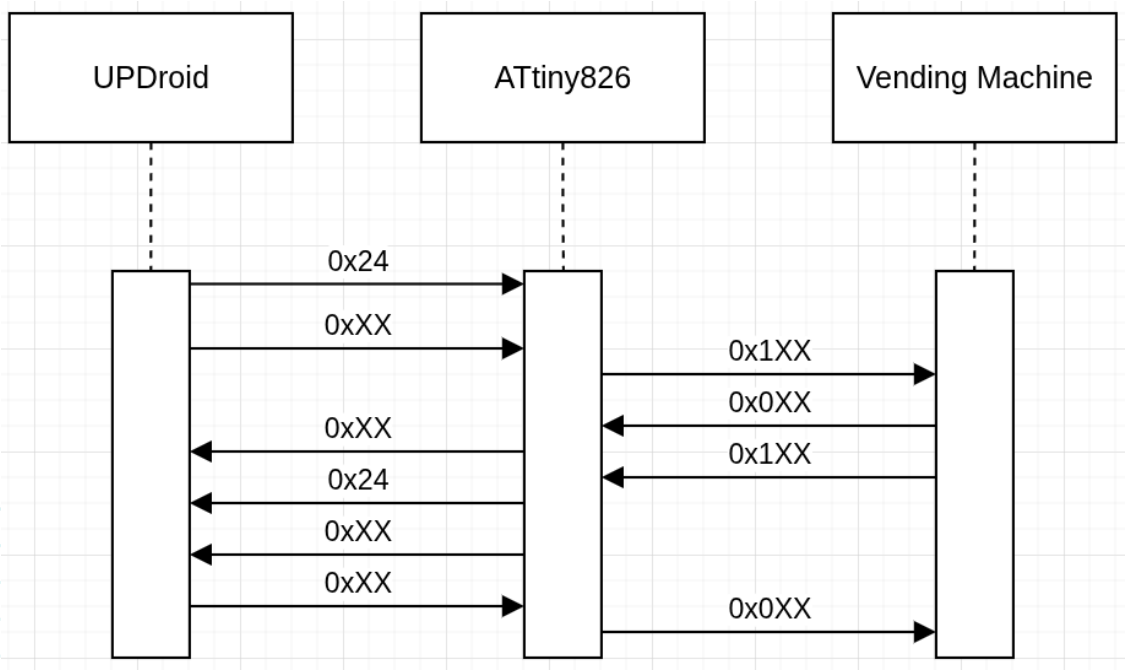

Fig. 1: MDB to Serial conversion

4.2. Display Interface

The Telit SE250B4 module provides one LCM interface, which complies to MIPI DSI standard. The interface supports high-speed differential data transmission and support up to HD+ display (1680 x 720 at 60 fps).

The selected panel uses a FL7707N driver ic, the relevant kernel overlays and bootloader configurations files for this interface can be seen below.

File	Path	Function
dsi-panel-fl7707n-djn-720x720-video.dtsi	LA.UM.9.15.2/LINUX/android/vendor/qcom/proprietary/devicetree-4.19/qcom	Contains the mdss panel node for the display, contains the majority of the required configurations.
scuba-sde-display.dtsi	LA.UM.9.15.2/LINUX/android/vendor/qcom/proprietary/devictree-4.19/qcom	Contains the configuration of the default panel in the sde_dsi node, and add the necessary power supplies properties.
scuba-iot-idp.dtsi	LA.UM.9.15.2/LINUX/android/vendor/qcom/proprietary/	Contains redundant panel configuration.

File	Path	Function
	devicetree-4.19/ qcom	
LCMDetectLib.c	BOOT.XF.4.1/ boot_images/ QcomPkg/SocPkg/ AgattiPkg/Library/ MDPPlatformLib	Contains the boot-loader program that detects which type of panel is connected at boot time.

4.3. Touchscreen Interface

The Telit SE250B4 module provides one I2C interface exclusive to Touch Panels and also provides the corresponding power supply and interrupt pins. The IC selected for driving the touch panel is the Goodix G911, the relevant kernel overlays and configuration file for this interface can be seen below.

File	Path	Function
tg2-touch.dtsi	LA.UM.9.15.2/LINUX/ android/vendor/qcom/ proprietary/device-tree-4.19/qcom	Contains the touch-screen driver compati-ble node inside the i2c node.
scuba-pinctrl.dtsi	LA.UM.9.15.2/LINUX/ android/vendor/qcom/ proprietary/devic-tree-4.19/qcom	Contains the touch-screen pins in the ts_int nodes
gtxx_v2.8	LA.UM.9.15.2/LINUX/ android/kernel/ msm-4.19/drivers/in-put/touchscreen	Contains the touch-screen driver folder.

4.4. NFC Interface

The Telit SE250B4 module provides one SPI interface exclusive to the NFC interface and also provides the corresponding power supply and interrupt pins. The IC selected for NFC card emulation and reading is the Renesas PTX130R, the relevant kernel overlays and configuration files for this interface can be seen below.

File	Path	Function
scuba.dtsi	LA.UM.9.15.2/LINUX/ android/vendor/qcom/ proprietary/device-tree-4.19/qcom	Contains the nfc dri-ver node in the qupv3_se0_spi spi in-terface.
scuba-pinctrl.dtsi	LA.UM.9.15.2/LINUX/ android/vendor/qcom/	Contains the nfc irq pins in the ptxnfc_irq node.

File	Path	Function
	proprietary/device-tree-4.19/qcom	
gtxx_v2.8	LA.UM.9.15.2/LINUX/android/vendor/ptx	Contains the driver vendor hals and folder.

4.5. Coin Pulse Interface

The module have four GPIO used to generate and capture pulse signals, each signal have a specific pin, which can be seen bellow.

SIGNAL	Logic Level	GPIO	Direction
PULSE05	Active low	52	output
PULSE50	Active low	47	output
PULSEPROD	Active low	96	input
PULSECOIN	Active low	49	input

To read or write to each signal, a Android framework API is available through the **TelitManagerService** where an GPIO API is available, for a usage example see the **TelitManagerDemo** application and the **ITelitManager.aidl** interface file.

4.6. Module power state management

An Attiny 826 Microcontroller manages the ON/OFF state of the SE250B4 module, by default the CBL_PWR_N pin stays grounded and makes the module stay always turned on. It's possible to configure the Microcontroller to be turned on and off by a switch connected between TP125 and ground, to activate this configuration define the macro **DEMO_BOARD** when compiling the attiny source code.

Another use for the TP125 pin is for tampering detection, by default this pin is configured as an input with internal pull-up, if connected to ground the Microcontroller activates the anti-tampering mode, should the ground ever be removed while in this mode the module will be locked on an off state and will only work again if it's EEPROM is cleared.

SIGNAL	Logic Level	Function
ON_OFF2SE	Active low	Trigger module ON/OFF state
RST2SE	Active low	Trigger the module RESET
CBL_PWR_N	Active low	Initiate power-on when grounded
TP125	Active low	May be used to detect tampering

4.6.1. Flashing the attiny

The ATtiny826 microcontroller uses an Unified Program and Debug Interface (UPDI) to flash its memory, the only physical connection required is the UPDI pin, Power (VCC) and Ground (GND). The code was developed in Arduino IDE

for easy of use, the following configuration is required to add the board to the board manager, and to set the required fuses for flashing.

Inside Arduino IDE:

1) Add Board Manager URL
   a) Go to File > Preferences
   b) In the Additional board manager URL field, paste the following URL:

```
http://drazzy.com/package_drazzy.com_index.json
```

2) Install megaTinyCore
   a) Go to Tools > Board > Boards Manager
   b) In the search bar, select *megatinycore* and click install.
3) Upload the firmware
   a) Go to Tools > Board and scroll down to the *megatinycore* section.
   b) Select the ATtiny3226/1626/826/426/3216/1616/816/416/1606/806/406 option.
   c) Under Tools, configure the following settings

```
Chip: ATtiny826
Clock: 16 MHz internal
Programmer: jtag2updi (if using an Arduino) or SerialUPDI - SLOW
```

   a) Connect your programmer to the computer
   b) Select the correct PORT for your board under the Tools menu.
   c) Click Sketch > Upload Using Programmer